Esercitazione 5 aa25/26

Fit non lineari, istogrammi, uso di ROOT come macro o libreria

Versione 1.4 - 2025-11-16

Corso di Laboratorio 1 - modulo di calcolo
Laurea in Fisica, Università di Genova

Questa esercitazione tratta: fit di grafici con funzioni non lineari, manipolazione dei dati in ingresso per la creazione del grafico, uso della libreria OO (Object-Oriented) ROOT, creazione e fit di istogrammi.

L'esercizio 1 affronta un semplice fit non lineare aggiungendo output di varia natura: parametri e loro errori, chi-quadro, etc.
L'esercizio 2 richiede di creare un programma compilato che fa uso delle librerie di ROOT per la manipolazione dei dati in ingresso durante la lettura dei file di input e la realizzazione di un fit non lineare relativamente complesso.
L'esercizio 3 richiede di realizzare un fit della funzione esponenziale ad una distribuzione di dati rappresentata con un istogramma.

Per la consegna degli esercizi potete usare il form Aulaweb associato a questa esercitazione.

Durata esercitazione: 3.5 ore - usate questo tempo in modo efficace, cercando di arrivare in fondo all'esercitazione. Provate a risolvere da soli gli esercizi proposti, ma se avete difficoltà non esitate a chiedere il supporto dei docenti.

Esercizio 1

Un termistore è un sensore la cui resistenza ha una forte dipendenza dalla temperatura:

$$R(T) = R_0 exp(\frac{T_0}{T})$$

in cui la temperatura T e il parametro T_0 sono in gradi Kelvin.

Si crei una macro di ROOT (esecizio1.C) che faccia un fit dei dati contenuti nel file **termistore.dat** e che stampi i valori di R_0, T_0, del chi quadro, del numero di gradi di libertà e della probabilità del chi quadro. Ogni riga del file **termistore.dat** corrisponde ad una misura ed ha il formato:

$$T \quad R \quad \sigma_T \quad \sigma_R$$

Istruzioni e commenti

- Si crea un oggetto TGraphErrors tramite il costrutture che utilizza il nome del file di input come argomento, cioè:

  ```
  TGraphErrors graph("file_di_dati.dat");
  ```

  In questo modo le quattro colonne del file saranno associate alle caratteristiche *x, y, x_err,* ed *y_err* per ogni punto del grafico.

- I parametri del modello e le rispettive incertezze possono essere ottenuti, dopo aver effettuato il fit, con i comandi GetParameter() e GetParError(). Se ad esempio si ha un oggetto chiamato `fun` della classe `TF1`, il parametro 3 della funzione, e la sua incertezza si ottengono con le istruzioni:

  ```
  double p3 = fun.GetParameter(3);
  double p3err = fun.GetParError(3);
  ```

- Il chi quadro, il numero di gradi di libertà e la probabilità del chi quadro si ottengono rispettivamente con i metodi `GetChisquare()`, `GetNDF()` e `GetProb()` di `TF1`. Il significato statistico di queste grandezze verrà affrontato nel corso di laboratorio. Se ad esempio si ha un oggetto chiamato `fun` della classe `TF1`, allora si può scrivere:

  ```
  double prob = fun.GetProb();
  double chi2 = fun.GetChisquare();
  int ndf = fun.GetNDF();
  ```

Materiale da consegnare: esercizio1.C

Esercizio 2

Un sensore produce un segnale in tensione V, da cui è possibile risalire alla temperatura T:

$T = a + bV$

in cui *a* e *b* sono due coefficienti di calibrazione. Si sa che la temperatura di un campione ha un andamento sinusoidale in funzione del tempo:

$T(t) = T0 + A \sin(\omega t + \varphi)$

Si consideri il file **dati_es2.txt**, con il seguente formato:

```
a b e_a e_b
t1 V1 e_t1 e_V1
 .  .  .    .
 .  .  .    .
 .  .  .    .
tn Vn e_t1n e_Vn
```

in cui la prima riga contiene i coefficienti di calibrazione e le relative incertezze, e le righe successive, il cui numero non è noto a priori, contengono coppie di valori (t, V) con le relative incertezze. I valori di tempo sono espressi in secondi e le tensioni in V. I coefficienti *a* e *b* sono espressi rispettivamente in C e C/V.

Creare un programma compilato che:

- Legga i coefficienti di calibrazione dal file di input.
- Calcoli per ogni punto la temperatura corrispondente con l'incertezza associata.
- Produca un grafico di T in funzione di t, in cui sugli assi coordinati compaiano i nomi e le unità di misura delle quantità graficate, e in cui i punti del grafico siano rappresentati come dei triangolini.
- Realizzi un fit con la funzione sinusoidale descritta in precedenza, facendo in modo che la funzione venga visualizzata nel grafico di colore blu o verde.
  Vedere di seguito le istruzioni per l'inizializzazione del fit.
- Stampi su terminale valori e incertezze dei parametri T0, A e ω ottenuti dal fit.

Si consiglia di scaricare da aulaweb il codice sorgente iniziare (esercizio2.cpp), che include già le intestazione necessarie alla compilazione e la creazione di una istanza dell'oggetto TApplication, utilizzato per mantenere attiva l'interfaccia grafica durante l'esecuzione di un programma compilato.

Nota 1: l'incertezza eT sulla temperatura si ottiene con la propagazione delle incertezze:

$$T = a + b \cdot V, \quad \text{quindi} \quad eT = e_a + e_b \cdot V + b \cdot e_V$$

Nota 2: in questo esercizio non è possibile creare l'oggetto `TGraphErrors` passando direttamente nel costrutture il nome del file di input poiché dobbiamo fare delle operazioni sui valori di input di ogni punto. Si dovrà quindi:

- Aprire il file tramite un oggetto ifstream per poi ed eseguire un ciclo while per la lettura del contenuto
- All'interno del ciclo while, effettuare le manipolazioni necessarie dei dati di input e poi chiamare i metodi `SetPoint(int i, double x, double y)` e `SetPointError(int i, double x_err, double y_err)`, per aggiungere i punti al grafico.

Nota 3: dato il file esercizio2.cpp, il comando di compilazione è il seguente:
g++ -o esercizio2 esercizio2.cpp -Wall -pedantic $(root-config --cflags --libs)
oppure, per tcsh:
g++ -o esercizio2 esercizio2.cpp -Wall -pedantic `root-config --cflags --libs`

Materiale da consegnare: esercizio2.cpp

Suggerimenti per l'inizializzazione del fit dell'esercizio 2

Nell'esercizio 2 si deve eseguire un fit su dei dati modellizzati con la funzione

$T(t) = T0 + A \sin(\omega t + \varphi)$

Si tratta di un segnale sinusoidale di ampiezza A che oscilla attorno a un valore di riferimento T0, con pulsazione ω e fase φ. Il valore da usare per l'inizializzazione di ciascuno di questi parametri di fit può essere stimato grossolanamente a partire dal grafico dei dati. Più in particolare:

- T0 corrisponde al valore attorno a cui oscilla la sinusoide
- A è la semidifferenza tra il massimo e il minimo valore sull'asse y
- $\omega = 2\pi/P$, dove P, il periodo dell'oscillazione, può essere stimato dalla distanza lungo l'asse x tra due massimi successivi
- Se $t=-\varphi/\omega$ allora $T(t=-\varphi/\omega) = T0$, quindi si cerca un valore di tempo t^* per cui T valga circa T0, e da lì si ricava il valore di inizializzazione $\varphi^* = -\omega t^* = -2\pi t^*/P$. Come primo tentativo si può anche lasciare questo parametro non inizializzato, per poi inizializzarlo con il criterio descritto solo nel caso in cui questo passaggio si riveli effettivamente necessario.

Esercizio 3

Un muone è una particella instabile che decade secondo una legge probabilistica per cui, dato un numero di particelle iniziali N_0, il numero di particelle che sopravvive al tempo t, $N(t)$, è legato ad esso tramite la seguente equazione:

$$N(t) = N_0 exp(\frac{-t}{\boldsymbol{\tau}})$$

In cui la costante $\boldsymbol{\tau}$ viene denominata vita media, o tempo di decadimento medio.

Il file **decay.dat**, allegato a questa esperienza, contiene 1000 misure del tempo di decadimento di un muone in secondi.

Creare un programma che:

- Inizializzi un oggetto di tipo TH1D - i.e. istogramma della libreria ROOT - con un numero adeguato di bin nell'intervallo delle misure. Si suggerisce un numero di bin compreso tra 10 e 20, e un intervallo con *t_min = 0 s,* e *t_max* di alcuni micro-secondi.
  NB: ricordarsi di utilizzare subito il metodo `TH1D::Sumw2()` per abilitare gli errori nell'istogramma appena creato.
- Legga una ad una dalle righe del file decay.dat, contenente i tempi di decadimento misurati, e riempia l'istogramma tramite il metodo `TH1D::Fill(double x)`.
- Crei una funzione di fit utilizzando la classe TF1 per l'equazione della legge di decadimento con i due parametri liberi N_0 e $\boldsymbol{\tau}$.
- Disegni l'istogramma (con titoli adeguati degli assi) e ne esegua un fit in modo da estrarre N_0 e $\boldsymbol{\tau}$.
- Infine stampare su schermo il valori di $\boldsymbol{\tau}$ ed N_0 con gli errori associati. Ragionare sul significato fisico di N_0.

L'esercizio può essere sviluppato, a vostra scelta, utilizzando una macro di ROOT (esecizio3.C) ***oppure*** un codice compilato che utilizzi le classi di ROOT (esercizio3.cpp).

Materiale da consegnare: esercizio3.C ***oppure*** esercizio3.cpp